VENTURE VISION UBUNTU

TRUST RUNTIME · VERIFICATION STATE SPACE

VVU-VAL-001
72-Hour Validation

Pre-Registration Protocol · v1.1 Reviewer-Revised · Published Before T=0

Venture Vision Ubuntu
Epistemic Runtime · 72-Hour Continuous Validation

Commit hash (frozen): To be published prior to production release
Dress rehearsal: REQUIRED before public run
Independent observers: invited before T=0

This protocol is frozen at publication.
Public run is executed against one frozen build;
subsequent runs are separately versioned validation events.

1. Primary Objective

Produce a public, independently reviewable validation artifact demonstrating that the Epistemic Runtime maintains its documented guarantees under progressively degrading operating conditions over 72 continuous hours. The experiment is successful only if the runtime continues satisfying its invariants, or fails according to documented policy (for example, transitioning to DEGRADED or FAIL-CLOSED rather than silently corrupting state).

> ***This is a pre-registration.*** *The test plan, success criteria, failure schedule, and Validation Index formula below are frozen and published before T=0. The frozen Git commit hash is recorded in §2.* ***The public validation event is executed against one frozen build. Any subsequent execution constitutes a separately versioned validation event*** *(VVU-VAL-002, VVU-VAL-003, etc.), with its own pre-registration and commit hash. If the runtime fails publicly under the published protocol, the failure, the logs, the artifacts, and the postmortem are published alongside any subsequent successful rerun.*

2. Frozen Artefacts

The following artefacts are frozen before the public run commences. The dress rehearsal (§9) must pass end-to-end before these are frozen. Once frozen, the public run is executed against these exact values; no changes are permitted. A subsequent validation event requires a new version number and a new pre-registration.

Artefact	Value	Status
Protocol version	VVU-VAL-001 v1.1	Frozen at publication
Git commit hash	To be published prior to production release (set by rehearsal harness at freeze)	To be set
Failure schedule	validation/chaos/schedule.yaml (6 phases, gate-mapped)	Frozen
Success criteria + severity levels	§3 below	Frozen
Validation Index formula + weights + threshold	§7 below	Frozen
Threat model (validated vs not validated)	§8 below	Frozen
Evidence bundle format	§6 below	Frozen
Independent observer attestation format	§10 below	Frozen
Dress rehearsal	Private run, must pass before public run (§9)	Required

Table 2.1. Frozen artefacts. The commit hash is the single value filled in after the dress rehearsal passes.

3. Success Criteria

The runtime must demonstrate the following across the full 72-hour run. Each criterion is independently verifiable from the evidence bundles (§6). Deviations are classified by severity, with each severity carrying a defined consequence.

- **Deterministic replay** — the replayed Fact Log produces the same checksum as the live log at every hourly checkpoint.
- **Append-only Fact Log** — no Fact is modified or deleted once accepted.
- **Deterministic HLC merge** — the post-partition merge (§4, P7) produces zero conflicts observed, zero data loss observed.
- **MMR integrity** — the Merkle Mountain Range root is valid at every checkpoint and identical whether computed from the live or replayed log.
- **Policy enforcement** — every policy violation produces a Failure Fact (not a crash); the runtime halts or degrades per documented policy.
- **Replay reproducibility** — any reviewer can re-run the replay against the published evidence bundle and obtain the same result (procedure in §12).
- **No silent corruption** — any state divergence is detected and logged; no silent data loss is observed.
- **Complete observability** — every minute's metrics, logs, and hashes are archived in the evidence bundles.

3.1 Severity Levels

Every deviation from the success criteria is classified into one of three severity levels. Each severity has a defined consequence, so the run's outcome is deterministic given the observed deviations.

Severity	Definition	Examples	Consequence
Critical	A violation of a core invariant that the validation is designed to prove	Silent Fact Log corruption; Fact modified after acceptance; MMR root mismatch between live and replay; accepted spoofed payload (HF-001 bypass)	Run terminates immediately; overall outcome FAIL; postmortem required before any subsequent validation event
Major	A deviation that degrades the run but does not violate a core invariant	Circuit Breaker FAIL-CLOSED outside P5/P6; data loss in NATS queue; replay divergence that self-corrects; policy violation not handled per spec	Run continues; -10 points from the Validation Index; documented in Final Report; does not by itself cause FAIL
Minor	A deviation that does not affect invariants or data integrity	Latency p99 above target; transient pod restart; log gap < 60s; cosmetic dashboard issue	Warning only; run continues; documented in Final Report; no index penalty

Table 3.1. Severity levels. A single Critical failure causes overall FAIL regardless of the Validation Index score. Major failures accumulate as index penalties. Minor failures are warnings only.

3.2 PASS / FAIL Determination

The overall outcome is determined by two independent conditions, both of which must be satisfied for a PASS:

- **Condition 1 (no Critical failures):** zero Critical-severity deviations observed across the full 72 hours.
- **Condition 2 (index threshold):** final Validation Index ≥ 90.0 at H72.

A PASS requires both conditions. A FAIL results from either a single Critical failure or a final index below 90.0. The threshold rationale is in §7.4.

4. Failure-Injection Schedule (Gate-Mapped)

The 72-hour run is divided into 6 phases (plus a recovery step). Each phase maps directly to a documented Hard Failure gate or chaos-engineering level, so the test exercises each claim in the Epistemic Runtime specification. The schedule is published verbatim in validation/chaos/schedule.yaml; the chaos driver applies the injections at the scheduled hour.

Phase	Hours	Gate	Injected	PASS criteria
P1 Nominal	0–12	Baseline	Normal traffic at 50 payloads/s	CB NORMAL; 100% throughput; zero rejected Facts
P2 Flood	12–24	Acceptance Capacity	Ramp 10× → 25× → 50× → 100×	Queue absorbs; no corruption; CB stays NORMAL/DEGRADED
P3 Network Chaos	24–36	HLC Ordering	Packet loss 5–40%; latency 50–2000ms; dup/drop	Replay deterministic; HLC order preserved
P4 Storage Pressure	36–48	Append-Only Integrity	Disk fill 70→95%; IO throttle	Graceful degradation; Fact Log append-only; MMR valid
P5 Node Failure	48–60	Recovery	Kill runtime/NATS/worker/API/scheduler randomly	Pods restart; no Fact loss; CB recovers
P6 Security	60–66	HF-001/002/005	Bad signatures; bad ZK; contradictory telemetry; clock skew; spoofed; replay; dup IDs	Every spoofed rejected (HF-001); bad ZK rejected (HF-002); contradictory halts (HF-005)
P7 Partition + Recovery	66–72	LVL-17 (72h Blackout)	Disconnect cluster; produce Facts; reconnect at H71	Deterministic HLC merge; zero corruption; replay identical; MMR identical

Table 4.1. The 6-phase gate-mapped failure schedule. Published before T=0; executed against one frozen build.

5. Infrastructure

> ***Iron rule:*** *the experiment must NOT depend on a workstation staying powered on. The runtime lives in infrastructure designed to survive laptop logout, OS updates, power loss, and SSD failure. The stream originates from the server, not a PC. The PC is only a viewer.*

The validation stack runs on a provider-agnostic k3s cluster (Hetzner / DigitalOcean / AWS / Azure / bare metal). Minimum host: 8 vCPU, 32 GB RAM, 250 GB NVMe, Ubuntu 24.04. The stack is split into 8 namespaces; the first 6 are Layer 1 (validation), the 7th is Layer 1 supporting (chaos injection), and the 8th is Layer 2 (outreach, killable). Every component writes to persistent volumes; nothing lives only in memory.

Namespace	Layer	Role	Killable?
vvu-runtime	1 (validation)	Epistemic Runtime, AIR Kernel, NATS, telemetry generator, security injector	No
epistemic	1 (validation)	Epistemic Runtime support services	No
air	1 (validation)	AIR Kernel support	No
evidence	1 (validation)	Hourly evidence archiver	No
monitoring	1 (validation)	Prometheus, Grafana	No
streaming	1 (validation)	Headless streaming service (Grafana dashboard → public stream)	No
chaos	1 (validation)	Chaos injection driver	No
outreach	2 (outreach)	Outreach engine (reads evidence bus)	Yes — killable without affecting the run

Table 5.1. Namespace layout. The streaming namespace runs a headless streaming service (implementation-agnostic — OBS, FFmpeg, or a cloud streaming API) that captures the Grafana dashboard and publishes to the public stream. The outreach namespace is the only killable one.

6. Evidence Bundles and Archival

Every hour, an immutable evidence bundle is produced: Hour-NN.zip. Each bundle contains logs, metrics, hashes, the MMR root, the fact count, the replay checksum, and system health. Each bundle is SHA-256 stamped; the stamp is appended to an append-only SHA256SUMS file.

```
VVU-72H-VALIDATION/ ├─ Hour-01.zip ... Hour-72.zip (hourly evidence bundles) ├─
SHA256SUMS (append-only hash ledger) ├─ Runtime Logs/ (all component logs) ├─
Metrics/ (Prometheus snapshots) ├─ Grafana JSON/ (dashboard exports) ├─ Replay
Dataset/ (replayable Fact Log) ├─ Fact Log/ ├─ MMR/ ├─ Root Hashes/ ├─ Screenshots/
(SIMULATION-labeled; see §6.1) ├─ Video/ (72h recording) ├─ Independent Observer
Attestations/ (see §10) ├─ Test Manifest ├─ Final Report.pdf └─
VVU-VAL-001_Pre_Registration_Protocol.pdf (this document)
```

6.1 Screenshot Labeling (Simulation vs Production)

> ***Simulation labeling is mandatory.*** *Any screenshot, image, or video frame originating from simulation mode (the deterministic 72-hour simulation driver used before the real run) must carry a visible "SIMULATION — NOT PRODUCTION EVIDENCE" watermark. Screenshots from the public run carry a "PRODUCTION EVIDENCE — VVU-VAL-001" label with the commit hash. This prevents any reviewer from mistaking a simulation screenshot for production evidence.*

6.2 Dual Archival: GitHub + Long-Term Preservation

GitHub is excellent for collaboration and immediate public access, but it is not a formal archival system. The evidence package is therefore archived in two locations:

- **GitHub Release** (immediate access, collaboration) — the complete VVU-72H-VALIDATION package is published as a GitHub release on the VVU repository, with the SHA256SUMS file in the release notes. This provides immediate, version-tagged public access.
- **Long-term archival** (preservation, immutability) — the same package is archived in an institutional repository or immutable object storage intended for long-term preservation. The planned archival target is Zenodo (which mints a DOI and provides a permanent URL) with a mirrored copy in an S3-compatible immutable bucket with object-lock enabled. The archival DOI is recorded in the Final Report and in the GitHub release notes.

The dual-archival approach ensures the evidence is both immediately accessible (GitHub) and durably preserved (Zenodo + immutable object storage). The SHA-256 of the complete package is identical across both archives; reviewers can verify either copy.

7. Validation Index (Published Formula)

The VVU Validation Index (0–100) replaces the earlier arbitrary constant with a value derived entirely from observable system properties. The formula, weights, measurement rules, and threshold rationale are published here so the index is reproducible, not a black box. The index is computed every minute from real measurements and displayed on the public Mission Control scoreboard.

> *Index = Σ ($weight_i$ × $dimension_i$) for i in dimensions; weights sum to 1.0; each dimension is measured 0–100 from real telemetry.*

Dimension	Weight	Measurement	Formula
Replay Determinism	0.20	Replay checksum matches Fact Log	100 if live == replay else 0
Evidence Integrity	0.20	Hourly bundles hash-verified	100 × verified / total
TEE Attestation	0.15	HF-001: spoofed payloads rejected	100 × (1 − accepted_bad / spoofed)
Policy Conformance	0.15	Violations handled per spec	max(0, 100 − 4 × unhandled)
Merge Correctness	0.15	HLC merges conflict-free	100 if 0 conflicts else max(0, 100 − 50×conflicts/merges)

Dimension	Weight	Measurement	Formula
Availability	0.15	Uptime minus FAIL-CLOSED time	100 × (1 − fail_closed_s / elapsed_s)

Table 7.1. Validation Index dimensions, weights, and measurement formulas. Weights sum to 1.0. Computed by validation/evidence/validation-index.py.

7.1 Weight Rationale

The weights reflect a deliberate engineering judgement about which dimensions are most load-bearing for the runtime's core value proposition (verifiable, append-only, replayable state). A reviewer is entitled to ask why Replay and Integrity are weighted at 0.20 while the other four are at 0.15:

• **Replay Determinism (0.20)** — the single most load-bearing property. If the Fact Log cannot be replayed deterministically, every other guarantee is moot: the MMR root, the ZK proofs, the on-chain anchors all derive from the replayed log. A replay failure is a Critical failure that undermines the entire trust chain. The 0.20 weight reflects this foundational status.

• **Evidence Integrity (0.20)** — the evidence bundles are the artefact that makes the validation independently reviewable. If the bundles themselves are not hash-verified, no reviewer can trust the rest of the index. The 0.20 weight reflects that the integrity of the evidence substrate underpins every other claim.

• **TEE Attestation, Policy Conformance, Merge Correctness, Availability (0.15 each)** — these four dimensions are each important but individually narrower in scope. TEE attestation covers the HF-001 gate only; policy conformance covers the policy engine's behaviour under stress; merge correctness covers the HLC merge at P7; availability covers uptime. Each is a necessary condition for the runtime's correctness, but none individually undermines the entire trust chain the way a replay or evidence-integrity failure would. The 0.15 weight reflects this narrower-but-necessary status.

These weights are an initial engineering baseline. The VVU team and the independent observers (§10) will review the weighting after the first public run; empirical calibration may justify adjustment in a subsequent validation event (VVU-VAL-002). The weights in this protocol are frozen; any change requires a new version number.

7.2 Threshold Rationale (PASS ≥ 90.0)

The PASS threshold of 90.0 is an initial engineering threshold, not a derived constant. The rationale:

• The threshold must be high enough that a PASS is meaningful — a runtime scoring 70/100 has demonstrated survival but not the level of correctness the trust-chain value proposition requires.

• The threshold must not be so high (e.g. 99.0) that normal operational variance (a single Major failure costing 10 points; transient availability dips in P5) makes PASS unattainable in practice. A 99.0 threshold would conflate 'perfect' with 'validated'.

• 90.0 is the midpoint that allows one Major failure (10-point penalty) while still permitting PASS, provided all other dimensions are near-perfect. This aligns with the severity model (§3.1): a single Major failure is recoverable; a Critical failure is not.

This threshold is an initial engineering baseline subject to future empirical calibration. After the first public run, the VVU team and independent observers will review whether 90.0 correctly distinguishes 'validated' from 'survived but degraded'. Any adjustment requires a new protocol version (VVU-VAL-002). The threshold in this protocol is frozen.

8. Threat Model — What This Validation Does and Does Not Prove

This section prevents readers from overextending the validation's conclusions. The 72-hour protocol validates specific engineering properties of the Epistemic Runtime under controlled failure injection. It does not validate properties that depend on real-world deployment conditions, sensor hardware, or adversarial conditions outside the published schedule.

8.1 Validated by this protocol

Property	How validated	Phase
Deterministic replay of the Fact Log	Replay checksum vs live checksum at every hourly checkpoint	All phases
Append-only Fact Log integrity	No Fact modified or deleted after acceptance; MMR root valid	All phases
HLC merge correctness under partition	P7 partition + reconnect; zero conflicts observed	P7
MMR integrity under stress	MMR root valid at every checkpoint; identical live vs replay	All phases
Policy enforcement under degradation	Every violation produces a Failure Fact; no crash	P2–P6
TEE attestation rejection (HF-001)	Every spoofed payload quarantined at Pass 2	P6
ZK proof rejection (HF-002)	Every bad ZK proof rejected; no WRT minted	P6
Decision derivation halt (HF-005)	Contradictory telemetry halts inference; TRIP verdict	P6
Recovery from node failure	Pods restart; no Fact loss; CB recovers	P5
Evidence bundle integrity	Hourly bundles SHA-256 verified	All phases

Table 8.1. Properties validated by this protocol.

8.2 NOT validated by this protocol

Property	Why not validated	Where it would be validated
Municipal hydraulics accuracy	No real water-network telemetry; synthetic payloads only	Municipal pilot (separate programme)
Sensor accuracy (acoustic leak detection)	No physical Hydro-Gateway hardware; simulated telemetry	Hardware prototype validation + municipal pilot
Production cybersecurity (penetration testing)	Controlled security injection only (P6); no adversarial red-team	Independent security audit (planned, separate)
Manufacturing reliability (24 parametric constraints)	No fabricated hardware; specification only	Fabricator FAI + production-quality audit
Long-term durability (10-year field life)	72-hour run only; no accelerated life testing	Field deployment + multi-year observation
Federation correctness (multi-org)	Single-writer ledger only; federation is v1.2	VVU-VAL-002+ after v1.2 release
ZK proof system soundness (production trusted setup)	Test-only trusted setup in use	Production trusted-setup ceremony (separate)
On-chain proof registry (Polygon mainnet)	Polygon Amoy testnet only	Post-pilot mainnet migration
Human-factors / operator UX	No human operators in the loop during the run	Municipal pilot + operator training
Regulatory compliance certification	No audit; policies are Draft for Adoption	ISO 27001 / SOC 2 audits (separate)

Table 8.2. Properties NOT validated by this protocol. Reviewers should not extend the validation's conclusions to these areas.

> ***Reading guidance:*** *a PASS on this protocol means the Epistemic Runtime's core engineering guarantees (replay, append-only, merge, MMR, policy, HF gates) hold under the published failure schedule. It does NOT mean the Hydro-Gateway detects real leaks, that the hardware survives field deployment, that the system is secure against red-team attack, or that the runtime is certified for production. Each of those is a separate validation programme.*

9. Public Mission Control Scoreboard

A public scoreboard is visible to everyone for the full 72 hours. It displays: elapsed time, current phase, Circuit Breaker state, facts accepted / queued / merged / rejected, policy violations, CPU, RAM, queue depth, latency p99, proof count, TEE status, replay status, the MMR root, the Validation Index with its 6 dimensions, the event log (live signed milestone events), the MMR root progression, the hourly evidence-bundle grid, and the overall PASS / FAIL banner. The scoreboard reads from a metrics endpoint that, in production, aggregates the evidence bus. In the current build it runs in simulation mode (a deterministic 72-hour simulation driver) so it is demonstrable before the real run.

> ***Simulation labeling:*** *when the scoreboard is running in simulation mode, the top banner displays a visible "SIMULATION MODE — NOT PRODUCTION EVIDENCE" indicator. When the scoreboard is running against the public validation, the banner displays "PRODUCTION EVIDENCE — VVU-VAL-001 — commit ". This prevents any reviewer from mistaking a simulation screenshot for production evidence (§6.1).*

The scoreboard does NOT send any outreach; it is purely a viewer. Outreach is handled by the separate, killable Layer 2 outreach engine (§11).

10. Independent Observers

Independent observers are the single biggest credibility opportunity for this protocol. Their role is narrow and well-defined: they do not endorse the system, and they do not assess whether the runtime is 'good'. They only attest that the published artifacts match what they observed during the run. This attestation is what moves the validation from a self-reported demonstration toward an independently corroborated engineering validation.

10.1 Observer Categories

Category	Who	Attestation scope
Academic	A faculty member from a computer-science or distributed-systems department (Wits / UCT / other)	Verify timestamps, hashes, and replay; publish a short attestation letter
Industry	A senior engineer from a distributed-systems or blockchain-adjacent company	Verify timestamps, hashes, and replay; publish a short attestation letter
Community	A recognized member of the open-source distributed-systems community	Verify timestamps, hashes, and replay; publish a short attestation letter

Table 10.1. Independent observer categories. Observers are invited before T=0; their identities and attestation methodology are published in the Final Report.

10.2 Observer Responsibilities

Each independent observer is asked to perform the following during and after the run:

• **During the run:** monitor the public scoreboard at irregular intervals; record the MMR root and Validation Index they observe; note any discrepancy between the scoreboard and their own independent query of the runtime's API.

• **After the run:** download the published evidence package; independently verify the SHA-256 of the package against the SHA256SUMS ledger; independently run the replay verification (§12) against a sample of hourly bundles; confirm that the replayed checksums match the published checksums.

• **Attestation:** publish a short letter (1 page) stating whether the published artifacts match what they observed. The letter does not assess the runtime's quality; it only attests to the match between observed and published data. Letters are published in the Independent Observer Attestations

directory of the evidence package.

10.3 Attestation Format

Each observer's attestation letter follows a standard format:

```
INDEPENDENT OBSERVER ATTESTATION VVU-VAL-001 Observer: <name, affiliation, category>
Observation period: <start> to <end> 1. Timestamps observed: <list of checkpoints
where the observer recorded the MMR root and Validation Index> 2. Hash verification:
<SHA-256 of the published evidence package matches the SHA256SUMS ledger? YES / NO>
3. Replay verification: <the observer independently ran the replay verification
procedure (§12) against N hourly bundles. The replayed checksums matched the
published checksums? YES / NO> 4. Discrepancies: <any discrepancy between observed
and published data, or 'none observed'> 5. Attestation: <'I attest that the published
artifacts match what I observed during the run, to the best of my ability.' OR a
specific description of any discrepancy.> Signature: <digital signature> Date: <date>
```

Observers are not compensated and are not asked to endorse the system. Their attestation is a factual statement about artifact integrity, not a quality judgement. The VVU team publishes the attestation letters verbatim, including any that report discrepancies.

11. Dress Rehearsal (Required)

> ***Before going public, a private dress rehearsal is mandatory.*** *The rehearsal executes the complete 72-hour protocol privately (in compressed time), fixes every issue discovered, and repeats until the run completes successfully. Only then is the codebase frozen, the commit hash recorded in §2, and the public run scheduled.*

The rehearsal harness (validation/rehearsal/run-rehearsal.sh) spins up a local k3s cluster, deploys the validation stack, drives the 6-phase failure-injection schedule in compressed time (≈2 minutes wall-clock for 72 simulated hours), archives hourly evidence bundles, runs replay verification after each phase, and reports PASS / FAIL with the Validation Index. If the rehearsal fails, the root cause is fixed and the rehearsal is re-run. The public run is executed against one frozen build.

12. Independent Reproduction

The strongest evidence this protocol can produce is not the VVU team's own validation run — it is an independent engineer's ability to reproduce the result. The following 8-step procedure lets any reviewer clone, verify, and reproduce the validation against the frozen build. If an independent engineer completes these steps and obtains the same outputs, the protocol moves beyond a demonstration toward a reproducible engineering validation.

12.1 Reproduction Procedure

```
STEP 1. Clone the repository git clone https://github.com/vvu/epistemic-runtime.git
cd epistemic-runtime STEP 2. Check out the frozen commit git checkout
<frozen-commit-hash> # from §2 of this protocol git rev-parse HEAD # confirm it
matches the published hash STEP 3. Download the evidence package # From the GitHub
Release (immediate access) wget
https://github.com/vvu/epistemic-runtime/releases/download/\
vvu-val-001/VVU-72H-VALIDATION.zip # OR from the long-term archival DOI (in the Final
Report) # Verify the SHA-256 matches the published SHA256SUMS ledger sha256sum
VVU-72H-VALIDATION.zip grep VVU-72H-VALIDATION.zip SHA256SUMS STEP 4. Run replay
verification against a sample of hourly bundles for HOUR in 12 24 36 48 60 66 72; do
bash validation/evidence/verify-replay.sh \ --bundle VVU-72H-VALIDATION/Hour-$(printf
%02d $HOUR).zip done # Expected: each verification prints 'VERIFICATION PASS' STEP 5.
Compare replay checksums # The replayed Fact Log checksum must match the recorded
checksum # in each bundle's metrics/replay-checksum.txt for HOUR in 12 24 36 48 60 66
72; do cat VVU-72H-VALIDATION/Hour-$(printf %02d $HOUR)/metrics/replay-checksum.txt
done # Each must match the live checksum produced in STEP 4 STEP 6. Compare MMR roots
# The replayed MMR root must match the recorded MMR root for HOUR in 12 24 36 48 60 66
72; do python3 -c "import json; print(json.load(open(
'VVU-72H-VALIDATION/Hour-$(printf %02d $HOUR)/metrics/ledger-status.json'
))['mmr_root'])" done # Each must match the live MMR root produced in STEP 4 STEP 7.
Compare the Validation Index # Recompute the Validation Index from the recorded
metrics python3 validation/evidence/validation-index.py \ --metrics
VVU-72H-VALIDATION/Hour-72/metrics/ledger-status.json # The recomputed index must
match the published final index STEP 8. Confirm identical results # If STEP 4 through
STEP 7 all produce identical results to the # published artifacts, the validation is
independently reproduced. # Publish your reproduction (blog post, GitHub issue, or
letter) so # other reviewers can corroborate.
```

12.2 Reproduction Expectations

If the independent reproduction produces identical results (replay checksums match, MMR roots match, Validation Index matches), the reviewer has independently confirmed the validation. If the reproduction produces different results, the reviewer has discovered a discrepancy that the VVU team must address in a postmortem. Either outcome is valuable evidence; the protocol is designed to make both outcomes possible.

Reviewers are encouraged to publish their reproduction results, whether confirming or discrepant. The VVU team commits to addressing every published discrepancy in a public response, and to re-running the validation if a discrepancy is confirmed.

13. Two-Layer Architecture

The validation (Layer 1) and the outreach (Layer 2) are strictly separated. The runtime emits signed milestone events to an evidence bus (NATS / Kafka). A separate, killable outreach pipeline consumes those events. Outreach crashes never affect the validation. If the outreach namespace is deleted, the 72-hour run continues unaffected; the evidence bundles still archive hourly.

```
72-Hour Runtime (Layer 1) | ▼ Evidence Event Bus (NATS / Kafka) | ├─ Dashboard
Updates ├─ Headless Streaming Service ├─ Archive Publisher (Layer 1) └─ Outreach
Engine (Layer 2 — killable)
```

14. Staged Release

Outreach is staged so the evidence leads the conversation. A mass-blast to all audiences at once is structurally impossible: the staged-release enforcement (validation/outreach/stages.yaml) locks each audience behind a stage gate.

Stage	Trigger	Audiences	Cooldown
1. Evidence Publication	M72 milestone (validation complete)	Technical communities, researchers	Immediate
2. Personalized Outreach	Stage 1 complete + 24h elapsed	Investors, municipalities (tailored emails referencing the published evidence by hash)	24 hours
3. General Social & Press	Stage 2 complete + 48h elapsed	Journalists, general social (concise posts linking back to the evidence)	48 hours

Table 14.1. Staged release. The outreach engine refuses to send to any audience not in the current stage's allowed list.

> ***What we will NOT do:*** *automate unsolicited outbound email to hundreds of recipients, or mass-post across platforms immediately after a successful run. The staged sequence lets the evidence lead the conversation and gives every recipient a concrete, reproducible artifact to inspect rather than a marketing claim.*

15. Operator Responsibilities

The protocol is machine-focused, but the run is not operator-free. A human operator is on-call for the full 72 hours to handle hardware replacement and to respond to Critical failures. To close the avenue of criticism that 'an operator could have intervened to fix the run', the operator's responsibilities are tightly constrained and fully logged.

15.1 Operator Constraints

Rule	Rationale
No code changes	The frozen commit hash (§2) must remain the build under test for the full 72 hours
No configuration edits	Configuration changes could alter the runtime's behaviour mid-run, invalidating the earlier phases
No manual Fact Log edits	The Fact Log is append-only and immutable; any edit is a Critical failure (§3.1)
No manual Circuit Breaker transitions (except documented recovery)	The CB must transition per the state machine; manual transitions are permitted only for the documented P5/P7 recovery sequences, and are logged + signed
Manual intervention only for hardware replacement	If a physical node fails, the operator may replace it; the replacement is logged with timestamp, node ID, and operator signature
All interventions logged	Every operator action — SSH session, kubectl command, hardware touch — is logged to an append-only operator log
All interventions signed	Every log entry is signed with the operator's Ed25519 key; the operator's public key is published before T=0
Operator may NOT touch the evidence bundles	The evidence bundles are produced by the archiver and are immutable once written; the operator has no write access

Table 15.1. Operator constraints. The operator log is published as part of the evidence package and is subject to independent observer review (§10).

15.2 Operator Log Format

```
OPERATOR LOG — VVU-VAL-001 Operator public key: <Ed25519 public key, published before T=0> [2026-XX-XX T+HH:MM:SS] [signed] Operator SSH session opened reason: <one-line description> [2026-XX-XX T+HH:MM:SS] [signed] kubectl command: <command> reason: <one-line description> [2026-XX-XX T+HH:MM:SS] [signed] Hardware replacement: node-3 reason: NVMe failure; replaced with identical model new serial: <serial> [2026-XX-XX T+HH:MM:SS] [signed] Operator SSH session closed
```

The operator log is part of the evidence package and is reviewed by the independent observers (§10). Any operator intervention that violates the constraints in Table 15.1 is a Critical failure and causes overall FAIL.

16. What We Will Not Do

• We will NOT depend on a workstation staying powered on — the runtime lives in infrastructure designed to survive laptop logout, OS updates, and power loss.

• We will NOT make the public stream the primary evidence — the stream is supporting evidence; the real evidence is the immutable logs, metrics, replay package, and hashes.

• We will NOT deliberately destroy infrastructure for dramatic effect — every failure injection maps directly to a documented guarantee.

• We will NOT define an arbitrary validation constant — the Validation Index is derived from real measurements via the published formula in §7.

• We will NOT run the public event before the dress rehearsal passes end-to-end.

• We will NOT change the protocol, schedule, success criteria, weights, or threshold after T=0 — the pre-registration is frozen; any change requires a new protocol version (VVU-VAL-002).

• We will NOT allow operator intervention that violates the constraints in §15 — any violation is a Critical failure.

• We will NOT publish simulation screenshots as production evidence — simulation outputs are watermarked (§6.1).

• We will NOT extend the validation's conclusions to properties it does not test (§8) — municipal hydraulics, sensor accuracy, production cybersecurity, and manufacturing reliability are separate validation programmes.

• We will NOT mass-blast outreach — the staged-release enforcement (§14) makes it structurally impossible.

17. Limitations

One frozen build: *the public validation is executed against one frozen build. A subsequent validation event requires a new protocol version and a new pre-registration. A failure is itself valuable evidence — the logs, artifacts, and postmortem are published alongside any subsequent successful rerun.*

Controlled failures: *the failure injections are controlled, not field. Real municipal outages may have different characteristics (duration, partial vs total, recovery pattern) that affect the runtime differently. Field validation is the subject of the municipal pilot, not this protocol.*

Validation Index is a snapshot: *the index reflects the behaviour observed during the 72-hour run under the published schedule. It is not a guarantee of future behaviour under different conditions. The weights (§7.1) and threshold (§7.2) are initial engineering baselines subject to empirical calibration in subsequent validation events.*

ZK-proof system: *the runtime uses a test-only trusted setup during validation. A production trusted-setup ceremony is planned before mainnet deployment; validation results do not extend to the production setup.*

Independent observers are attestors, not endorsers: *the observers (§10) attest that the published artifacts match what they observed. They do not assess whether the runtime is 'good' or fit for any particular purpose. Their attestation is about artifact integrity, not system quality.*

Threat model scope: *the validation proves the properties in Table 8.1 and nothing else. The properties in Table 8.2 (municipal hydraulics, sensor accuracy, production cybersecurity, manufacturing reliability, long-term durability, federation, production ZK setup, mainnet registry, human factors, regulatory certification) are NOT validated by this protocol and are each the subject of a separate validation programme.*

18. Publication Checklist (Before T=0)

- Dress rehearsal passes end-to-end (validation/rehearsal/run-rehearsal.sh).
- Codebase frozen; commit hash recorded in §2.
- This protocol PDF (v1.1) published with the frozen commit hash.
- validation/chaos/schedule.yaml published.
- Validation Index formula, weights rationale (§7.1), and threshold rationale (§7.2) published.
- Threat model (§8) published.
- Independent observers invited; their identities and methodology published.
- Operator public key published; operator constraints (§15) acknowledged.
- Public Mission Control scoreboard reachable, with SIMULATION/PRODUCTION label working.
- Evidence bundle archival (validation/evidence/bundle.sh) verified.
- Dual archival targets configured (GitHub Release + Zenodo/immutable object storage).
- Outreach registries (milestones / recipients / stages) published.
- GitHub Actions workflows (validation-automation + outreach-engine) enabled.
- Independent Reproduction procedure (§12) tested by at least one internal reviewer.
- T=0 announced with the frozen commit hash and observer list.